Complete API Audit

Lead Integration & Distribution Middleware

Inventory · Gap Analysis · Security Audit · Architecture Review

Audit date: 2026-10-01 · Method: full-codebase inspection · No code was changed

HTTP routes	81 across 17 controllers — all under `/api/v1`
Guarding	73 permission-guarded · 6 public · 2 authenticated (self-scoped)
Statuses	77 IMPLEMENTED · 2 PARTIAL · 2 BROKEN
Security findings	10 total — 2 HIGH (cross-tenant queue reads, wildcard API keys)
Missing vs required	2 required endpoints (change-password, import cancel) + 9 recommended/planned
Frontend coverage	69/69 calls matched · 11 unconsumed endpoints · 4 mismatches
OpenAPI	None existed before this audit → generated at `docs/openapi.yaml`
HTTP tests	0 controller/guard tests exist (data-layer tests do)

Files: docs/api-inventory.md · docs/api-gaps.md · docs/api-security-audit.md · docs/api-architecture.md

Contents

API Inventory — Lead Integration & Distribution Middleware

A. IMPLEMENTED APIs

B. PARTIAL APIs

C. MISSING APIs

D. DUPLICATE APIs

E. UNUSED APIs (implemented, zero consumers)

F. BROKEN APIs

G. REQUIRED APIs (product-complete checklist)

H. SECURITY FINDINGS (summary)

I. FRONTEND / API MISMATCHES

J. RECOMMENDED API ARCHITECTURE

Appendix 1 — Conventions verified

Appendix 2 — Test coverage of API surface

API Gaps Analysis — Lead Integration & Distribution Middleware

1. Authentication
2. Tenants
3. Users
4. Roles & permissions
5. Leads
6. Lead sources
7. Webhooks
8. Destinations
9. Integrations (OAuth)
10. Field mapping
11. Routing rules
12. Scoring
13. Deduplication

14. Enrichment
15. Deliveries
16. Queue
17. CSV import
18. Audit logs
19. Analytics
20. Explicitly NOT required (with reasons)

Priority queue (for your approval)

API Security Audit — Lead Integration & Distribution Middleware

Findings (severity-ordered)

Verification matrix (per §10 checklist)

Explicitly tested behaviors that back these claims (existing test suite)

Recommended remediation order (approval required before any change)

API Architecture — Lead Integration & Distribution Middleware

1. System shape
2. Request lifecycle (contract)
3. Authentication model
4. Data & idempotency model
5. Queue topology (BullMQ)
6. Egress & adapters
7. Frontend ↔ API mapping (coverage health)
8. Testing posture
9. Documentation & OpenAPI posture
10. Recommended architecture (J)

API Inventory — Lead Integration & Distribution Middleware

Audit date: 2026-10-01 · **Method:** full-codebase inspection (controllers → services → Prisma, web queries, worker, tests) · **No code was changed.**

Metric	Value
HTTP routes	**81** across **17 controllers**
Global prefix / version	`/api/v1` (URI versioning, `apps/api/src/main.ts:49-50`)
Auth	Bearer JWT **or** `x-api-key` header (global `JwtAuthGuard`)
Permission-guarded routes	73 (`@RequirePermission`) · `@Public` 6 · authenticated-no-permission 2
Envelope	`{ success, data }` success · `{ success:false, error:{code,message,requestId,details?} }` error
Pagination	cursor (`take+1`) on 9 list endpoints; **6 unbounded lists** (finding)
OpenAPI artifact	**none existed** → generated at `docs/openapi.yaml` by this audit
Frontend consumers	69 distinct calls, all matched to real routes (0 calls to missing endpoints)

Status legend: `IMPLEMENTED` works end-to-end · `PARTIAL` exists but incomplete behavior · `BROKEN` exists but returns wrong/unsafe data · `†` implemented but has **no consumer** (see §E).

A. IMPLEMENTED APIs

auth (`apps/api/src/modules/auth/auth.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
POST	/api/v1/auth/register	auth	Public (20/min)	—	n/a (creates tenant)	IMPLEMENTED	Register workspace + admin + system roles in one tx; sets refresh cookie, returns tokens (201)
POST	/api/v1/auth/login	auth	Public (20/min)	—	via credentials	IMPLEMENTED	Email/password (+optional slug) → access+refresh; generic 401, audits failures

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
POST	/api/v1/auth/refresh	auth	Public (20/min)	—	via refresh token	IMPLEMENTED	Rotating refresh (cookie or body); Origin check; reuse detection revokes all sessions
POST	/api/v1/auth/logout	auth	Public	—	via refresh token	IMPLEMENTED	Revokes session + clears cookie; **no audit row** (finding H-7)
GET	/api/v1/auth/me	auth	Session	—	own tenant	IMPLEMENTED	Current user + role + permissions (bootstrap for web)

tenants (`tenants/tenants.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/tenants/me	tenants	Session	—	own tenant	IMPLEMENTED	Workspace name/slug/status/createdAt
PATCH	/api/v1/tenants/me	tenants	Session	tenant:write	own tenant	IMPLEMENTED	Rename workspace (slug immutable)

users (`users/users.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/users	users	Session/Key	users:read	yes	IMPLEMENTED	List users (cursor `limit≤100`, includes role + counts)
POST	/api/v1/users	users	Session/Key	users:write	yes (actor)	IMPLEMENTED	Create user with role (201; bcrypt hash never returned)
PATCH	/api/v1/users/:id	users	Session/Key	users:write	yes (check-then-act)	IMPLEMENTED	Update name/role/status (activate/deactivate; self-disable refused)

roles (`roles/roles.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/roles	roles	Session/Key	roles:read	yes	IMPLEMENTED	List roles with permission arrays + userCount (**unbounded**, finding H-8)
GET	/api/v1/roles/permissions	roles	Session/Key	roles:read	n/a (static)	IMPLEMENTED	Permission catalog (34 codes) + 4 role templates — serves the "GET /permissions" requirement
POST	/api/v1/roles	roles	Session/Key	roles:write	yes (actor)	IMPLEMENTED	Create custom role (201; permissions validated against catalog)
PATCH	/api/v1/roles/:id	roles	Session/Key	roles:write	yes	IMPLEMENTED	Update role; system roles immutable (409/403)
DELETE	/api/v1/roles/:id	roles	Session/Key	roles:write	yes	IMPLEMENTED	Delete; refused while assigned or system

api-keys (`api-keys/api-keys.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/api-keys	api-keys	Session/Key	apikeys:read	yes	IMPLEMENTED	List keys (prefix, status, lastUsed; hash never returned; cursor)
POST	/api/v1/api-keys	api-keys	Session/Key	apikeys:write	yes (actor)	IMPLEMENTED	Create key; raw value returned **once** (201). **Empty scopes ⇒ wildcard `*` admin key** (finding H-2)
DELETE	/api/v1/api-keys/:id	api-keys	Session/Key	apikeys:write	yes	IMPLEMENTED	Revoke (idempotent)

audit-logs (`audit-logs/audit-logs.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/audit-logs	audit-logs	Session/Key	audit:read	yes	IMPLEMENTED	Filter (action/entityType/entityId/actor/from/to), cursor; sanitized metadata, no secrets

leads (`leads/leads.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/leads	leads	Session/Key	leads:read	yes	IMPLEMENTED	List/filter (status, sourceId, destinationId, dedupStatus, date range), cursor + total
POST	/api/v1/leads	leads	Session/Key	leads:write	yes (actor)	IMPLEMENTED	Manual create → enqueues intake (201). **409** on duplicate (`tenant,sourceAccount,externalLeadId`); **no idempotency without externalLeadId** (finding H-5)
GET	/api/v1/leads/:id	leads	Session/Key	leads:read	yes (404 cross-tenant)	IMPLEMENTED	Full detail incl. source, duplicate link, deliveries, scores, stages
GET	/api/v1/leads/:id/timeline	leads	Session/Key	leads:read	yes	IMPLEMENTED	Immutable LeadStageEvent timeline
GET	/api/v1/leads/:id/payloads	leads	Session/Key	leads:read	yes	IMPLEMENTED	Raw inbound payloads (unsanitized — see H-9)
POST	/api/v1/leads/:id/replay	leads	Session/Key	leads:replay	yes	IMPLEMENTED	Re-run pipeline (201 `{replayed:true}`), audited
DELETE	/api/v1/leads/:id	leads	Session/Key	leads:delete	yes	IMPLEMENTED	Hard delete + audit; **no frontend consumer** (§E)

sources (`sources/sources.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/sources	sources	Session/Key	sources:read	yes	IMPLEMENTED	List sources (**unbounded**, finding H-8)
POST	/api/v1/sources	sources	Session/Key	sources:write	yes (actor)	IMPLEMENTED	Create source (201; tenant-unique code)
GET	/api/v1/sources/:id	sources	Session/Key	sources:read	yes	IMPLEMENTED	Detail; **no frontend consumer** (§E)
PATCH	/api/v1/sources/:id	sources	Session/Key	sources:write	yes	IMPLEMENTED	Update **name/config only** — no activation exists: `LeadSource` has no status column and `sourceUpdateSchema` has no status field (see §C)
DELETE	/api/v1/sources/:id	sources	Session/Key	sources:write	yes	IMPLEMENTED	Delete; blocked (409) while leads attached
GET	/api/v1/sources/:id/accounts	sources	Session/Key	sources:read	yes	IMPLEMENTED	Accounts + inbound webhook URLs (publicId)
POST	/api/v1/sources/:id/accounts	sources	Session/Key	sources:write	yes	IMPLEMENTED	Create account; **signing secret returned once** (201; auto-created for WEBHOOK/API/CUSTOM types)
PATCH	/api/v1/sources/accounts/:accountId	sources	Session/Key	sources:write	yes	IMPLEMENTED	Update account; **not nested under `:id`** (naming); **no consumer** (§E)
DELETE	/api/v1/sources/accounts/:accountId	sources	Session/Key	sources:write	yes	IMPLEMENTED	Delete account
POST	/api/v1/sources/accounts/:accountId/rotate-secret	sources	Session/Key	sources:write	yes	IMPLEMENTED	Rotate HMAC secret; new value returned once (201)

webhooks (`webhooks/webhooks.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
POST	/api/v1/webhooks/inbound/:publicId	webhooks	Public (600/min)	—	derived from publicId	IMPLEMENTED	Signed inbound lead ingress: HMAC-SHA256 over raw body (timing-safe), DB idempotency `@@unique(tenant,account,idempotencyKey)`, fast-ack 201, queue-fail → 503. **Returns 201 not documented 202** (H-6); unsigned accepted if account has no secret unless `requireSignature`
GET	/api/v1/webhooks/events	webhooks	Session/Key	sources:read	yes	IMPLEMENTED	Event log (status/account/cursor); **query not zod-validated** (inline type), **no consumer** (§E)
GET	/api/v1/webhooks/events/:id	webhooks	Session/Key	sources:read	yes	IMPLEMENTED	Single event detail; **no consumer** (§E)

destinations (`destinations/destinations.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/destinations/types	destinations	Session/Key	destinations:read	n/a (static)	IMPLEMENTED	7 destination types + config metadata
GET	/api/v1/destinations	destinations	Session/Key	destinations:read	yes	IMPLEMENTED	List (**unbounded**, finding H-8)
POST	/api/v1/destinations	destinations	Session/Key	destinations:write	yes (actor)	IMPLEMENTED	Create (201; tenant-unique name)
GET	/api/v1/destinations/:id	destinations	Session/Key	destinations:read	yes	IMPLEMENTED	Detail; credential metadata only (`stored:true`, never the value)
PATCH	/api/v1/destinations/:id	destinations	Session/Key	destinations:write	yes	IMPLEMENTED	Update config/status

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
DELETE	/api/v1/destinations/:id	destinations	Session/Key	destinations:write	yes	IMPLEMENTED	Delete (+ health row)
PUT	/api/v1/destinations/:id/credential	destinations	Session/Key	destinations:write	yes	IMPLEMENTED	Write-only secret (AES-256-GCM at rest)
DELETE	/api/v1/destinations/:id/credential	destinations	Session/Key	destinations:write	yes	IMPLEMENTED	Remove credential
GET	/api/v1/destinations/:id/health	destinations	Session/Key	destinations:read	yes	IMPLEMENTED	Last probe result; **no consumer** (§E)
POST	/api/v1/destinations/:id/test	destinations	Session/Key	destinations:test	yes	IMPLEMENTED	Real outbound probe through SSRF-safe adapter (201), audited

field-mappings (`field-mappings/field-mappings.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/field-mappings	field-mappings	Session/Key	mappings:read	yes	IMPLEMENTED	List (destinationId filter, cursor)
GET	/api/v1/field-mappings/:id	field-mappings	Session/Key	mappings:read	yes	IMPLEMENTED	Detail incl. all versions
POST	/api/v1/field-mappings	field-mappings	Session/Key	mappings:write	yes (actor)	IMPLEMENTED	Create + v1 (201; optional immediate activate)
POST	/api/v1/field-mappings/:id/versions	field-mappings	Session/Key	mappings:write	yes	IMPLEMENTED	Append immutable version (201)
POST	/api/v1/field-mappings/:id/activate	field-mappings	Session/Key	mappings:write	yes	IMPLEMENTED	Activate version (201; rollback = activate older)
DELETE	/api/v1/field-mappings/:id	field-mappings	Session/Key	mappings:write	yes	IMPLEMENTED	Delete mapping + versions

routing-rules (`routing-rules/routing-rules.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/routing-rules	routing-rules	Session/Key	routing:read	yes	IMPLEMENTED	Ordered rules + destinations (**unbounded**, H-8)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
POST	/api/v1/routing-rules	routing-rules	Session/Key	routing:write	yes (actor)	IMPLEMENTED	Create (201; priority, conditions, destination targets)
PATCH	/api/v1/routing-rules/:id	routing-rules	Session/Key	routing:write	yes	IMPLEMENTED	Partial update incl. `{enabled}` toggle; version bump
DELETE	/api/v1/routing-rules/:id	routing-rules	Session/Key	routing:write	yes	IMPLEMENTED	Delete

automation — scoring / dedup / enrichment (`automation/automation.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/automation/scoring	automation	Session/Key	scoring:read	yes	IMPLEMENTED	Rule-set versions list (**unbounded**, H-8)
POST	/api/v1/automation/scoring	automation	Session/Key	scoring:write	yes (actor)	IMPLEMENTED	Create new immutable version (201; siblings disabled)
PATCH	/api/v1/automation/scoring/:id	automation	Session/Key	scoring:write	yes	IMPLEMENTED	Enable/disable one set
GET	/api/v1/automation/dedup	automation	Session/Key	dedup:read	yes	IMPLEMENTED	Read config — **lazily creates default row on first GET** (unaudited write, H-7)
PUT	/api/v1/automation/dedup	automation	Session/Key	dedup:write	yes (key=tenantId)	IMPLEMENTED	Full-replace config (strategy, matchFields, enabled), audited
GET	/api/v1/automation/enrichment	automation	Session/Key	enrichment:read	n/a	**PARTIAL**	Honest status stub `{implemented:false, providers:[], note}` — enrichment stage runs SKIPPED in worker

deliveries (`deliveries/deliveries.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/deliveries/queue-stats	deliveries	Session/Key	deliveries:read	**NO (global)**	IMPLEMENTED	BullMQ counts per queue + DLQs — all tenants (aggregate visibility, H-1a)
GET	/api/v1/deliveries/dead-letter	deliveries	Session/Key	deliveries:read	**NO (global)**	**BROKEN**	DLQ jobs incl. `job.data` (tenantId, leadId, payload) of **every tenant** → cross-tenant leak (H-1)
GET	/api/v1/deliveries/failed	deliveries	Session/Key	deliveries:read	**NO (global)**	**BROKEN**	Failed queue jobs incl. `job.data` — same cross-tenant leak (H-1). Query not zod-validated
GET	/api/v1/deliveries	deliveries	Session/Key	deliveries:read	yes	IMPLEMENTED	List (status/destinationId/**leadId**, cursor)
GET	/api/v1/deliveries/:id	deliveries	Session/Key	deliveries:read	yes	IMPLEMENTED	Detail + immutable `DeliveryAttempt` history (attemptNumber, replayCycle, replayCount, cycleStartAttempt)
POST	/api/v1/deliveries/:id/replay	deliveries	Session/Key	deliveries:retry	yes	IMPLEMENTED	Append-only replay: new cycle (+5 maxAttempts); **409** when PENDING/QUEUED/IN_PROGRESS/SUCCEEDED; 201 `{queued:true}`, audited

imports (`imports/imports.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
POST	/api/v1/imports	imports	Session/Key	imports:write	yes (actor)	IMPLEMENTED	multipart/csv upload (field `file`, 50 MB) + `sourceId`; enqueues `csv-import` (201); async
GET	/api/v1/imports	imports	Session/Key	imports:read	yes	IMPLEMENTED	Job list (status filter, cursor)
GET	/api/v1/imports/:id	imports	Session/Key	imports:read	yes	IMPLEMENTED	Job detail/progress — **returns `storageKey` + `checksum`** (H-9); **no consumer** (§E)
GET	/api/v1/imports/:id/errors	imports	Session/Key	imports:read	yes	IMPLEMENTED	CSV error report (`text/csv`, envelope bypassed via `@SkipTransform`, ≤10k rows); audited download

analytics (`analytics/analytics.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/analytics/overview	analytics	Session/Key	analytics:read	yes	IMPLEMENTED	DB-aggregated KPIs (counts, rates)
GET	/api/v1/analytics/timeseries	analytics	Session/Key	analytics:read	yes	IMPLEMENTED	Daily series (days 1–90, SQL group-by)
GET	/api/v1/analytics/sources	analytics	Session/Key	analytics:read	yes	IMPLEMENTED	Per-source performance (groupBy)
GET	/api/v1/analytics/destinations	analytics	Session/Key	analytics:read	yes	IMPLEMENTED	Per-destination performance — **loads all tenant delivery rows in JS** (H-8 hot spot)
GET	/api/v1/analytics/pipeline	analytics	Session/Key	analytics:read	yes	IMPLEMENTED	Stage funnel

health (`health/health.controller.ts`)

Method	Endpoint	Module	Auth	Permission	Tenant Scoped	Status	Purpose
GET	/api/v1/health	health	Public (300/min)	—	n/a	**PARTIAL**	Checks PG (`SELECT 1`) + Redis; **no worker check; returns HTTP 200 with `status:"degraded"` + raw driver errors when down** (H-10)

B. PARTIAL APIs

Endpoint	What works	What's missing
GET /api/v1/automation/enrichment	Honest `{implemented:false, note}` status	No providers, no config, no execution — worker stage is an explicit no-op (`apps/worker/src/pipeline/pipeline.service.ts:423-446`)
GET /api/v1/health	API+PG+Redis liveness with latency	No worker/BullMQ-consumer liveness; failure → HTTP 200 not 503; raw error strings leaked
POST /api/v1/imports (cancel half of the contract)	Worker honors `CANCELLED` status (`csv-import.processor.ts:41`), enum exists	**No `POST /imports/:id/cancel` endpoint** — half the stack supports it (see G)

C. MISSING APIs

Full analysis with rationale in `docs/api-gaps.md`. Summary:

Priority	Endpoint	Domain	Evidence it's needed
Required now	POST /api/v1/auth/change-password	auth	`changePasswordSchema` already shipped unused (`packages/validation/src/index.ts:283`)
Required now	POST /api/v1/imports/:id/cancel	imports	`CANCELLED` enum + worker check already exist
Required (schema first)	Source activation/deactivation (`status` on PATCH /api/v1/sources/:id)	sources	Requested capability; `LeadSource` lacks any status column — no endpoint or schema support exists
Recommended	GET /api/v1/users/:id	users	UI can only list; detail requires client-side scan
Recommended	DELETE /api/v1/users/:id	users	Only status-DISABLED exists today

Priority	Endpoint	Domain	Evidence it's needed
Recommended	GET /api/v1/audit-logs/:id	audit-logs	Pagination contract has no single-row fetch
Recommended	POST /api/v1/webhooks/events/:id/replay *or* documented provider-retry story	webhooks	FAILED events recoverable only by provider re-send
Planned	POST /auth/forgot-password, /auth/reset-password	auth	Needs mailer infra first (none exists)
Planned	OAuth authorize/callback/token-refresh per integration	integrations	Today: manually pasted static credentials only
Planned	Tenant CRUD (GET/POST/GET/PATCH/DELETE /tenants/:id) + activate/deactivate	tenants	Only `/tenants/me`; needed only for a super-admin console
Not required	Bulk import/update/replay, export, /leads PATCH, deliveries cancel, analytics/deliveries, analytics/queue, field-mapping preview/discovery, routing simulation, /health (unprefixed)	—	No architecture/UI consumes these; see api-gaps.md §"Explicitly not required"

D. DUPLICATE APIs

None found. Candidate pairs examined and confirmed distinct:

Candidate	Verdict
POST /leads/:id/replay vs POST /deliveries/:id/replay	**Not duplicates** — lead-level re-runs the whole pipeline; delivery-level appends an attempt cycle to one destination
/deliveries/failed (queue view) vs /deliveries?status=FAILED (delivery rows)	Different resources: BullMQ job state vs DB delivery rows. Both feed distinct UI pages
/roles/permissions vs a hypothetical /permissions	The catalog endpoint is the canonical permissions API; a second path would duplicate it
/sources/accounts/:accountId (flat) vs /sources/:id/accounts/:accountId (nested)	**Nesting inconsistency, not duplication** — see §Naming
/webhooks/inbound/:publicId vs domain's /webhooks/:source/:publicId	Single generic ingress by design; source is encoded in the account's publicId. Canonical as-is

Canonical choices to preserve: **replay** (never a separate `/retry` route), **/leads** plural, **/automation/{scoring,dedup,enrichment}** grouping.

E. UNUSED APIs (implemented, zero consumers)

Verified against apps/web (69 call templates), `scripts/*` (no API calls at all — only the local e2e sink server), and tests (no HTTP calls — services are driven directly). 11 routes have **no consumer anywhere**:

Endpoint	Likely reason	Verdict
POST /imports, GET /imports, GET /imports/:id, GET /imports/:id/errors	CSV upload UI never built	Keep — core product feature, wire UI later
GET /webhooks/events, GET /webhooks/events/:id	No event-log UI	Keep — add to Sources page later
DELETE /leads/:id	No delete affordance in lead UI	Keep — deliberate caution
GET /sources/:id	Web only lists + fetches accounts	Keep — cheap detail route
PATCH /sources/accounts/:accountId	Accounts are create/delete/rotate only in UI	Keep
GET /destinations/:id/health	UI uses `/test` instead	Keep — passive health vs active probe
POST /auth/register	No signup page (registration done via API/E2E)	Keep — bootstrap + future self-serve

Dead **frontend** code (not endpoints): `useCreateLead` (`apps/web/src/lib/queries/leads.ts:142`) and `useDelivery` (`queries/delivery.ts:37`) are never invoked; `requestBlob` and `webhookUrl` exports unused.

F. BROKEN APIs

Endpoint	Breakage	Evidence
GET /api/v1/deliveries/dead-letter	Returns `job.data` (tenantId, leadId, payload) for **all tenants** to any `deliveries:read` holder	`deliveries.service.ts:117-132`, `infra/redis.module.ts:215-256`
GET /api/v1/deliveries/failed	Same cross-tenant leak	same

Related-but-implemented (flagged, not broken): queue-stats is global by design of BullMQ counters (H-1a).

Contract deviations on working endpoints (documented in OpenAPI as-is): webhook ingress returns **201** where code comments say 202; lead replay returns `{replayed}` where web types say `{queued}` (§I).

G. REQUIRED APIs (product-complete checklist)

Domain	Required by product	Status
Auth: register/login/logout/refresh/me	yes	✅ implemented
Auth: change-password	yes	❌ missing (schema ready)
Auth: forgot/reset-password	only with email infra	⏸ planned

Domain	Required by product	Status
Tenants: me (+rename)	yes	✅
Tenants: CRUD/activate/limits/usage	only with admin console	⏸ planned
Users: list/create/update(activation,role)	yes	✅
Users: detail/delete	recommended	❌ missing
Roles: CRUD + permission catalog	yes	✅ (detail = list)
Leads: list/create/detail/timeline/payloads/replay/delete	yes	✅
Leads: filter/search/pagination/sort	yes	✅ (cursor + filters; no server sort — newest-first by design)
Sources: CRUD + accounts + webhook URL + secret rotation	yes	✅
Sources: activation/deactivation	yes (requested capability)	❌ MISSING — `LeadSource` has **no status column** (`schema.prisma:332`) and `sourceUpdateSchema` accepts only `name?/config?`; needs a schema addition first
Sources: test-connection/health	recommended	❌ missing (destinations have it; sources don't)
Webhooks: ingress + HMAC + idempotency + rate limit + event history	yes	✅ (201-vs-202 + optional-signature caveats)
Destinations: CRUD + credential + health + test	yes	✅
Integrations: OAuth flows	for hosted-credential product	⏸ planned (static credentials today)
Field mappings: CRUD + versions + activate	yes	✅ (validation/preview/discovery = optional)
Routing: CRUD + enable/disable	yes	✅ (validation/simulation = optional)
Scoring: versions + activate/enable + calc (worker)	yes	✅ (preview = optional)
Dedup: config + strategies + duplicate surfacing	yes	✅ (merge intentionally **not** supported — links, never merges)
Enrichment: providers/execution	only when provider exists	⏸ stub, honestly reported
Deliveries: list/detail/attempts/replay	yes	✅ replay semantics verified (append-only attempts, +5 cycles)
Deliveries: cancel	no (replay+DLQ model)	➖ not required
Queue: health/counts/failed/DLQ read	yes	⚠ implemented but BROKEN scoping (F)
Queue: admin ops (pause/flush/rotate)	no — must stay unexposed	➖ correctly absent
Imports: create/list/detail/errors + async processing	yes	✅
Imports: cancel	yes (enum+worker ready)	❌ missing endpoint

Domain	Required by product	Status
Audit: list (+filters/pagination)	yes	✅ (GET /:id optional)
Analytics: overview/leads(sources,destinations)/queue via overview+queue-stats	yes	✅ (dedicated /analytics/deliveries + /analytics/queue not needed)
Health: API/PG/Redis/Worker distinction, non-200 when degraded	yes	⚠ partial (H-10)

H. SECURITY FINDINGS (summary)

Full detail with evidence in `docs/api-security-audit.md`. Severity-ordered:

ID	Sev	Finding
H-1	**High**	Cross-tenant read: `/deliveries/dead-letter` + `/failed` return other tenants' `job.data` (also exposes tenantId/leadId); `/queue-stats` is global aggregate
H-2	**High**	`POST /api-keys` with default `scopes: []` ⇒ `permissions:["*"]` ⇒ bypasses every `@RequirePermission`
H-3	Med-High	Access tokens survive logout/revocation (guard never consults `UserSession`; exposure ≤ JWT_ACCESS_TTL 900 s)
H-4	Medium	Disabled user's API keys keep working (key path never loads owning user)
H-5	Medium	`POST /leads` ignores Idempotency-Key; without `externalLeadId` no dedup possible → duplicate rows on retry
H-6	Low-Med	Webhook ingress documented as 202, actually 201; unsigned traffic accepted when account has no secret (unless `requireSignature`)
H-7	Low	Unaudited writes: `GET /automation/dedup` lazy-create; `POST /auth/logout`
H-8	Medium	6 unbounded `findMany` lists (sources, source accounts, destinations, roles, routing-rules, scoring versions) + `analytics/destinations` loads every delivery row; 9 other lists correctly cursor-paginated
H-9	Low	`GET /imports/:id` leaks `storageKey` + `checksum`; refresh token returned in JSON body (also httpOnly cookie); `/imports/:id`… health exposes raw driver errors; `/leads/:id/payloads` unsanitized
H-10	Low	Health = HTTP 200 even when degraded; no worker liveness

Positives verified: tenant never taken from client input (credential-derived only); 73/78 non-public routes permission-guarded; secrets AES-256-GCM/bcrypt/SHA-256 at rest and select-filtered from responses; timing-safe webhook HMAC over raw body; SSRF-guarded egress with DNS-rebinding-safe private-IP checks; audit rows on ~40 mutation sites with secret stripping; no stack traces in responses; correlation IDs on every response.

I. FRONTEND / API MISMATCHES

#	Frontend believes	Backend does	Impact
1	`POST /leads` body `{sourceAccountId?, externalLeadId?, payload}` (`queries/leads.ts:145`)	`{sourceId, sourceAccountId (both required), externalLeadId?, data}` (`leads.dtos.ts:8-13`)	Would 400 — currently unreachable (hook is dead code)
2	Lead replay response `{queued:boolean}` (`queries/leads.ts:132`)	`{replayed:boolean}` (`leads.service.ts:250`)	Type-only today; value discarded by caller
3	"Retry at" column = next retry (`delivery/retry/page.tsx:58`, `queue-jobs-table.tsx:78`)	`retryAt` is filled from **job creation timestamp** (`infra/redis.module.ts:251`)	Misleading data on Retry page
4	—	—	Coverage itself is healthy: **69/69 calls match real routes**, no wrong HTTP methods, no calls to missing endpoints; all list envelopes `{items,nextCursor,total}` are supersets of what web reads

---

J. RECOMMENDED API ARCHITECTURE

Detailed in `docs/api-architecture.md`. Summary:

1. Keep the current shape: `/api/v1`, single Nest app, zod DTOs (`createZodDto`), `{success,data}` envelope, cursor pagination, guard chain. It is coherent and mostly complete — **fix, don't re-platform**.
2. Fix H-1/H-2 before any external exposure; add change-password + import cancel (schemas/enums already exist).
3. Introduce a `TenantQueueView` (filter job.data by actor tenant) or drop job payloads from queue endpoints; scope queue-stats per tenant.
4. Standardize list endpoints: either paginate the 6 unbounded lists or document them as bounded reference data with an explicit `take`.
5. Make webhooks honor 202 (or update the documented contract), and default new accounts to `requireSignature=true`.
6. Decide the missing-endpoint set from §C with product input; do **not** add OAuth/tenants-CRUD until their consumers exist.
7. Generate OpenAPI from the app (enable the already-installed `@nestjs/swagger` document builder — decorators are present but no document is registered) so `docs/openapi.yaml` becomes a build artifact, not hand-maintained.
8. Add HTTP-surface tests (supertest) for auth, RBAC, tenant isolation, idempotency, rate limits, 404/409 — today zero controller tests exist.

Appendix 1 — Conventions verified

- **Versioning:** 100% `/api/v1/...`; no unversioned or mixed paths (`main.ts:49-50`; no `@Version()` overrides).
- **Naming:** plural kebab-case resources everywhere (`/routing-rules`, `/field-mappings`, `/api-keys`); no verb-style routes. Inconsistencies: flat `/sources/accounts/:accountId` (vs nested siblings), `/automation/*` grouping vs top-level `/routing-rules`, `/tenants/me` instead of `/tenants/:id`.
- **Status codes:** Nest defaults (POST→201, others→200); no `@HttpCode` anywhere. 400 validation (zod via nestjs-zod), 422 service-level ZodError, 401/403/404/409 (Prisma P2002/P2025 mapped), 429 throttled, 503 queue-down, 500 generic. No 200-for-failure except health (H-10).
- **Idempotency-Key:** never read (grep=0). Real idempotency exists via DB uniques: webhook events, lead external ids, delivery identity, attempt numbers, outbound `Idempotency-Key` header to destinations.
- **Rate limits:** global 300/min; auth register/login/refresh 20/min; webhook ingress 600/min; nothing skipped.
- **Untyped public contracts (violations of §5):** `webhooks/events` query (inline TS type), `deliveries/dead-letter|failed` query params (raw strings), webhook ingress body (`unknown` — intentional), analytics handler DTOs inline. Everything else flows through zod DTOs.

Appendix 2 — Test coverage of API surface

- **HTTP-layer tests: zero** (`apps/api/src/**/*.spec.ts` = none; no supertest, no playwright despite `test:e2e` script).
- Indirect coverage exists: tenant-isolation (`tests/integration/tenant-isolation.test.ts`), DB idempotency (`idempotency.test.ts`), pipeline/replay/audit (`pipeline`, `replay`, `audit`), queue mechanics + signed-webhook e2e (`tests/queue/*`).
- Untouched by any test: auth, tenants, users, roles, api-keys, destinations, field-mappings, imports, analytics, health, automation endpoints, and all controllers' HTTP behavior (validation, status codes, rate limits).

API Gaps Analysis — Lead Integration & Distribution Middleware

Audit date: 2026-10-01 · Companion to `docs/api-inventory.md` §B/§C/§G. **Nothing has been implemented — this is the decision document for your review.**

Each gap states: what's asked for → what exists → what's actually needed for *this* product's architecture → recommendation. The guiding rule used throughout: **do not add an endpoint whose consumer (UI/worker/integration) does not exist in the architecture.**

---

1. Authentication

Requested endpoint	Exists?	Architecture reality	Recommendation
POST /api/v1/auth/register	✅	Creates tenant+admin+roles in one tx	Keep
POST /api/v1/auth/login	✅	JWT access (900 s) + rotating refresh cookie	Keep
POST /api/v1/auth/logout	✅	Revokes session server-side	Keep (add audit row — security H-7)
POST /api/v1/auth/refresh	✅	Rotation + reuse detection	Keep
GET /api/v1/auth/me	✅	Web bootstrap incl. permissions	Keep
POST /api/v1/auth/change-password	❌ **MISSING**	`changePasswordSchema` (12+ chars, upper/lower/digit) **already exported** at `packages/validation/src/index.ts:283` but no controller uses it; `userUpdateSchema` does not accept a password either	**REQUIRED** — half-built already; implement next
POST /api/v1/auth/forgot-password	❌	**No mailer/sms infra anywhere** (config has no SMTP/provider keys)	**PLANNED** — blocked on email infrastructure; a token would need a `PasswordResetToken` model (schema change)
POST /api/v1/auth/reset-password	❌	Same blocker	**PLANNED** with forgot-password

Gaps found beyond the requested list:

- Logout/refresh rotation never invalidates already-issued **access** tokens (see security H-3). No endpoint can fix this alone — needs guard-level session check or short-TTL discipline.
- No `POST /auth/sessions` (list active sessions) — optional; model `UserSession` already stores them.

---

2. Tenants

Requested	Exists?	Architecture reality	Recommendation
GET/POST /tenants, GET/PATCH/DELETE /tenants/:id	❌ (only `/tenants/me` GET+PATCH)	Tenants are created by **registration**; the product UI is single-workspace-per-login. There is no super-admin console, no cross-tenant operator	**NOT REQUIRED now.** If an admin console is planned: PLANNED
activate/deactivate tenant	❌	`Tenant.status` (SUSPENDED) exists and the guard already enforces 403 for suspended tenants (`jwt-auth.guard.ts:91`) — but nothing can set it	PLANNED with tenants-CRUD
settings	✅ via PATCH /tenants/me	name only; slug immutable	Enough for current UI
integrations / usage / limits	❌	No usage-metering or limit columns in schema	**NOT REQUIRED** — no billing/limits feature exists; do not invent

3. Users

Requested	Exists?	Notes	Recommendation
GET /users	✅ cursor	Includes role + counts	—
POST /users	✅	bcrypt, role assignment	—
GET /users/:id	❌	UI shows detail only via list scan (25/page)	**RECOMMENDED** — trivial, completes REST shape
PATCH /users/:id	✅	name/role/**status** (activate/deactivate built-in)	—
DELETE /users/:id	❌	Deactivation exists (DISABLED); hard delete doesn't	RECOMMENDED — decide product semantics: deactivation-only (GDPR soft) vs hard delete; audit row required
password (admin reset)	❌	—	Fold into change-password work (admin-set-temporary-password flow)
roles/permissions per user	✅	returned inside `/users` rows (role.permissions)	No separate `/users/:id/permissions` needed — `GET /auth/me` covers self, roles cover others

4. Roles & permissions

Requested	Exists?	Notes	Recommendation
GET/POST/PATCH/DELETE /roles	✅	System roles (admin/manager/developer/viewer) immutable; 409 when in use	—
GET /roles/:id	❌	List already returns full permission arrays for every role	**NOT REQUIRED** (list is the detail) — or add for REST purity
GET /permissions	✅ as `GET /roles/permissions`	34-code catalog + templates	Canonical as-is; don't duplicate path
GET /users/:id/permissions	❌	Redundant with role.permissions	NOT REQUIRED

5. Leads

Requested	Exists?	Notes	Recommendation
GET/POST/GET :id/DELETE :id	✅		—
PATCH /leads/:id	❌	Leads are pipeline-managed documents; the architecture never mutates lead content after intake (replay re-derives it)	**NOT REQUIRED** — adding it would break the immutability story
POST /leads/:id/replay	✅	leads:replay, audited, returns `{replayed}`	Fix web's `{queued}` type (mismatch I-2)
GET /leads/:id/timeline	✅	Stage events	—
GET /leads/:id/deliveries	✅ effectively	`GET /leads/:id` embeds deliveries; `GET /deliveries?leadId=` exists as a dedicated filter (`deliveryListQuerySchema` line 68)	NOT REQUIRED as separate path
GET /leads/:id/audit	✅ effectively	`GET /audit-logs?entityId=:id` (web already does this, `queries/leads.ts:121`)	NOT REQUIRED as separate path
bulk import	✅ via /imports	CSV path is the bulk path	—
bulk update / bulk replay	❌	No UI concept	NOT REQUIRED
search	⚠ partial	Filter-based (status/source/destination/date) but **no free-text search**	Decide: if lead UI needs keyword search, add `q` param to existing list (server-side ILIKE over canonical fields) — RECOMMENDED small enhancement, no new endpoint

Requested	Exists?	Notes	Recommendation
export	❌		NOT REQUIRED (imports error-CSV precedent exists if needed)
sorting	⚠	Newest-first fixed	Acceptable; add `order` only if users ask

6. Lead sources

Requested	Exists?	Notes	Recommendation
CRUD	✅	`PATCH /sources/:id` accepts **name/config only** (`sourceUpdateSchema`, `validation/index.ts:195-198`)	—
source activation/deactivation	❌ **MISSING (capability does not exist)**	`LeadSource` has **no status column** (`prisma/schema.prisma:332`) — no endpoint, no field, nothing to activate/deactivate. (The `status: ACTIVE|DISABLED` enum belongs to **destinations**, `validation/index.ts:215`.)	**REQUIRED per product domain spec — needs schema change first:** add `LeadSource.status`, accept it in `PATCH /sources/:id`, audit the toggle. Reported only; not implemented (audit rule + schema-change prohibition)
source accounts (credentials)	✅	create/list/delete + `rotate-secret`; secrets returned once, AES-GCM at rest	—
webhook configuration	✅	per-account `publicId` → `POST /webhooks/inbound/:publicId`; URL shown in `GET /sources/:id/accounts`	—
source health / test connection	❌	Destinations have `/health` + `/test`; sources have neither. Inbound sources don't need outbound probes, but a **webhook-received-last / failure-count** health view could reuse `WebhookEvent` stats	OPTIONAL — if built, prefer read-only computed health over a new probe endpoint
Google Ads / Meta / LinkedIn-specific handlers	❌	Architecture has **one generic signed ingress**, no per-provider verification (grep: no META/GOOGLE signatures)	PLANNED only if those integrations become a product goal — each needs provider-specific signature verification + payload normalizer, not just an endpoint
CSV as source	✅ via /imports	`sourceId` required on upload	—

7. Webhooks

Requested	Exists?	Notes	Recommendation
POST /webhooks/:source/:publicId	✅ as `/webhooks/inbound/:publicId`	Source is encoded in the account (publicId maps to source+tenant) — the `:source` segment would be redundant	Canonical as-is
registration	✅	Creating a source account registers the webhook (publicId issued)	—
verification	✅	HMAC-SHA256 over **raw body**, timing-safe compare, header auto-detect (`x-signature` / `x-hub-signature-256` / `x-middleware-signature`)	Fix H-6: enforce signature when secret exists; default `requireSignature=true` for new accounts
secret management	✅	returned once at create; `rotate-secret` endpoint	—
event history	✅	`GET /webhooks/events[/:id]` (filter status/account/cursor) — **no UI consumer yet**	Wire UI later; add zod DTO for the inline query type
failed webhook events	⚠	`FAILED` events exist; **resume path already recovers them when the provider retries** (duplicate → reset to RECEIVED)	Document "provider retry = replay". Optional internal `POST /webhooks/events/:id/replay` if operators want manual replay — RECOMMENDED only if event UI ships
rate limiting	✅	600/min per IP on ingress	—
payload validation	⚠	Body is `unknown` by design (raw bytes needed for HMAC); shape extracted opportunistically (event-id/provider-lead-id headers+body probing)	Acceptable; document expected payload contract in OpenAPI

8. Destinations

Requested	Exists?	Notes	Recommendation
CRUD + activate/deactivate	✅	`status` in update schema; `DestinationStatus` enum	—
credentials	✅	PUT/DELETE `/destinations/:id/credential`, write-only, AES-GCM	—
health	✅	`GET /:id/health` (stored probe) — no UI consumer	Wire UI later
test connection	✅	`POST /:id/test` real SSRF-guarded probe, audited	—

Requested	Exists?	Notes	Recommendation
configuration by type	✅	`GET /destinations/types` + type-specific zod config	—
Salesforce/HubSpot/Zoho/Pipedrive/DCT-CRM/webhook/custom REST	✅ adapters	All 7 `DestinationType`s implemented in `packages/integrations`	—

9. Integrations (OAuth)

Requested	Exists?	Architecture reality	Recommendation
OAuth authorize / callback / token refresh endpoints	❌	Credentials are **manually pasted static secrets**; adapters use stored values. No OAuth client registrations, no redirect URIs, no token vault columns for provider access/refresh tokens (only `IntegrationCredential.secret` single blob)	**PLANNED** — significant work: new models (provider auth code, access+refresh token, expiry), callback route (must escape `/api/v1` semantics or live under it), state/CSRF, refresh scheduling. Do not scaffold until a hosted-credential UX is approved
integration status/disconnect	✅ effectively	credential presence (`stored:true`) + health/test	Sufficient today
"never expose credentials in responses"	✅ verified	Selects exclude secrets; only `stored:true` flags	—

10. Field mapping

Requested	Exists?	Notes	Recommendation
GET/POST/DELETE /field-mappings	✅	tenant-unique name per destination	—
GET /field-mappings/:id	✅	incl. versions	—
PATCH /field-mappings/:id	❌	**By design**: edits append an immutable version (`POST /:id/versions`) + `POST /:id/activate`	NOT REQUIRED — versioning supersedes PATCH; document rationale
mapping validation	⚠	Server validates field schema (zod `fieldMappingFieldSchema`, 13 transforms) but no "dry-run against a sample lead"	OPTIONAL preview endpoint
mapping preview	❌		PLANNED (nice UX, needs sample payload input)
versions / publish	✅	create-version + activate (=publish), rollback = activate older	—

Requested	Exists?	Notes	Recommendation
destination field discovery	❌	Would require provider metadata APIs per adapter	PLANNED with OAuth work

11. Routing rules

Requested	Exists?	Notes	Recommendation
GET/POST/PATCH/DELETE /routing-rules	✅	priority-ordered; partial PATCH supports `{enabled}` = enable/disable	—
GET /routing-rules/:id	❌	List returns complete rows	NOT REQUIRED (list = detail)
rule validation	✅ at write	zod `routingRuleUpsertSchema` + condition/operator enum checks	—
rule ordering	⚠	`priority` on create/update; **no reorder endpoint** (swap via PATCH priorities)	Optional `PATCH /routing-rules/order` only if UI builds drag-and-drop
rule testing / preview / simulation	❌	Worker has `conditions.spec.ts`-verified evaluator that could be reused	OPTIONAL — a `POST /routing-rules/simulate {lead}` read-only evaluator would be genuinely useful; PLANNED

12. Scoring

Requested	Exists?	Notes	Recommendation
rules + configuration	✅	`GET/POST /automation/scoring`, `PATCH /:id` enable	—
score calculation	✅ in worker	pipeline SCORING stage writes immutable `LeadScore` rows	—
score versions	✅	immutable rule sets, `@@unique([tenantId,version])`, exactly one enabled	—
activation/deactivation	✅	PATCH `{enabled}` (server enforces single-enabled via sibling disable)	—
score preview	❌	Same optional simulate concept as routing	PLANNED (one shared `/pipeline/simulate` could cover scoring+routing+dedup)

13. Deduplication

Requested	Exists?	Notes	Recommendation
config + strategies	✅	`GET/PUT /automation/dedup`, 5 strategies (`MatchedFields`), custom matchFields ≤10	—
duplicate search	✅ effectively	`GET /leads?dedupStatus=…` + `GET /leads/:id` embeds `duplicateOf`	—
duplicate relationships	✅ stored	`LeadDuplicate` rows (strategy, confidence, originalLeadId) shown in lead detail	—
merge / review	❌	**Product deliberately links, never merges** ("duplicates kept and linked — never silently dropped")	**NOT REQUIRED** — do not invent merge semantics. If review workflows are wanted later: a `LeadDuplicate.status (REVIEWED/DISMISSED)` field would be the minimal additive change (schema change — not in this audit)
lazy-create side effect	⚠	`GET /automation/dedup` writes the default row on first read (unaudited, H-7)	Fix behavior, not the endpoint

14. Enrichment

Requested	Exists?	Notes	Recommendation
providers / configuration / execution / retry / health	❌ (status stub only)	Worker stage writes `{skipped:true, reason:"No enrichment provider is configured…"}` and marks stage **SKIPPED** — honest, tested behavior (`pipeline.service.ts:423-446`); API returns `implemented:false`	**PLANNED as a whole domain.** Required endpoints when a provider is chosen: `GET/PUT /automation/enrichment` (config), provider health within status, execution stays in the worker stage, retry rides existing lead replay. Keep the honest stub until then

15. Deliveries

Requested	Exists?	Notes	Recommendation
GET /deliveries, GET /:id	✅	detail embeds immutable `DeliveryAttempt[]`	—

Requested	Exists?	Notes	Recommendation
POST /:id/replay	✅	append-only: global attempt numbering, `replayCycle`, +5 `maxAttempts` per cycle, 409 guards, audited — regression-tested (`tests/integration/replay.test.ts`)	—
GET /deliveries/dead-letter	⚠ BROKEN	cross-tenant leak (F/H-1)	**Fix scoping before anything else**
retry	✅ = replay	Canonical single mechanism	Never add a second `/retry` route
cancel	❌	Replay + DLQ model has no cancel concept; a cancel would need worker cooperation for in-flight jobs	**NOT REQUIRED** (revisit if users ask to stop a queued delivery)
attempts / timeline / failure reason / destination response	✅	in detail payload (attempt rows carry status, error, response snippet, replayCycle)	—
queue status	⚠	see Queue section	Fix scoping

16. Queue

Requested	Exists?	Notes	Recommendation
GET /api/v1/health	✅ PARTIAL	PG+Redis only, 200-when-down, no worker	**REQUIRED fix:** add worker/BullMQ liveness, return 503 on failed critical checks (or document degraded contract loudly)
GET /health (unprefixed)	❌	No deploy manifests exist yet (no Docker/k8s)	Add only when a deployment target needs a probe path
GET /deliveries/queue-stats	✅	waiting/active/completed/failed/delayed per queue + DLQs	Scope or explicitly document as operator-global (H-1)
active/failed/delayed job reads	✅	`/deliveries/failed`, `/deliveries/dead-letter`	Fix scoping (H-1)
queue admin (pause/flush/purge)	❌ **intentionally**	"Never expose unsafe queue administration to normal users"	**Must remain absent** from the public API

17. CSV import

Requested	Exists?	Notes	Recommendation
POST /imports	✅	multipart, 50 MB cap, async enqueue, tenant-validated source	—
GET /imports, GET /:id	✅	status counters/progress in detail	—
POST /imports/:id/cancel	❌ **MISSING**	`ImportJobStatuses` includes `CANCELLED`; worker already aborts on it (`csv-import.processor.ts:41`) — **only the endpoint is absent**	**REQUIRED** — cheapest gap on this list; also audit the cancellation
upload validation/preview	⚠	Validation happens in the worker (row errors persisted); **no pre-flight preview**	Optional: `POST /imports/preview` (first N rows parsed synchronously) — PLANNED
result download	✅	`GET /:id/errors` CSV (≤10k rows, audited)	—
retry failed import	❌	Re-upload is the current path (deterministic externalLeadId makes re-upload idempotent)	Acceptable; optional `POST /:id/retry` later

18. Audit logs

Requested	Exists?	Notes	Recommendation
GET /audit-logs	✅	filters: action/entityType/entityId/actor/from/to; cursor; metadata sanitized (secret-stripping)	—
GET /audit-logs/:id	❌		OPTIONAL (list+entityId filter suffices)
verification checklist	✅/⚠	tenant-scoped ✅, pagination ✅, actor ✅, action ✅, entity ✅, timestamp ✅, correlationId ✅. Secrets: `sanitizeDeep` strips token/secret/password keys ✅	Watch: `HttpException` messages pass through verbatim — review before adding user-facing error messages

19. Analytics

Requested	Exists?	Notes	Recommendation
GET /analytics/overview	✅	SQL aggregation	—

Requested	Exists?	Notes	Recommendation
GET /analytics/leads	✅ effectively	overview + `timeseries?days=` + pipeline funnel	Dedicated path NOT REQUIRED
GET /analytics/sources	✅	groupBy	—
GET /analytics/destinations	✅	**implementation loads all tenant deliveries into JS** (H-8 hot spot)	Fix implementation (SQL group-by), not the contract
GET /analytics/deliveries	❌	Succeeded/failed counts live in overview	NOT REQUIRED
GET /analytics/queue	❌	Covered by `/deliveries/queue-stats`	NOT REQUIRED
browser-side computation	✅ verified	All aggregates are DB-side	—

20. Explicitly NOT required (with reasons)

Candidate	Why not
Tenant CRUD + usage/limits	No admin console, no billing/metering feature
OAuth endpoints	Manual-credential architecture; needs data model first
Lead PATCH / bulk ops / export	Pipeline-immutability design; no UI
Merge duplicates	Product links, never merges (tested invariant)
Deliveries cancel / queue admin	Replay+DLQ model; safety requirement to NOT expose
/users/:id/permissions, /roles/:id	Data already embedded in list/me responses
/leads/:id/deliveries, /leads/:id/audit	Covered by embedded detail + `?leadId=` / `?entityId=` filters
Unprefixed /health	No deployment target yet
Forgot/reset password	Blocked on nonexistent mail infrastructure
Per-provider webhook handlers (Meta/Google/LinkedIn)	Product currently promises one generic signed endpoint

Priority queue (for your approval)

1. **Security fixes first (no new endpoints):** H-1 queue tenant scoping, H-2 API-key wildcard scopes, H-4 disabled-user keys, H-6 signature default.
2. **Two cheap required endpoints:** `POST /auth/change-password` (schema ready), `POST /imports/:id/cancel` (enum+worker ready).

3. **Contract cleanups:** webhook 202-vs-201 decision, health 503/worker check, `retryAt` semantics, dead web hooks (`useCreateLead`/`useDelivery`), untyped query params (zod for `/webhooks/events`, `/deliveries/failed|dead-letter`).
4. **REST completeness:** `GET/DELETE /users/:id`, `GET /audit-logs/:id` (optional).
5. **Pagination hardening:** bound the 6 unbounded lists; SQL-ify `analytics/destinations`.
6. **Planned domains (separate approvals):** password reset (needs mailer), OAuth, enrichment providers, tenants-CRUD admin console, import preview, pipeline simulate.

API Security Audit — Lead Integration & Distribution Middleware

Audit date: 2026-10-01 · Scope: `apps/api/src` (guards, filters, all modules), `packages/{shared,validation,types,config}`, `prisma/schema.prisma`, egress adapters. Every finding cites code. **Nothing was changed.**

Severity: **HIGH** = exploitable now by a legitimate low-privilege tenant/user · **MEDIUM** = requires specific conditions or widens blast radius · **LOW** = hygiene/defense-in-depth.

Findings (severity-ordered)

H-1 · HIGH — Cross-tenant read via queue endpoints

`GET /api/v1/deliveries/dead-letter` and `GET /api/v1/deliveries/failed` return raw BullMQ job records **including** `job.data` for **every tenant's queues** to any principal holding `deliveries:read`.

- Evidence: `apps/api/src/modules/deliveries/deliveries.controller.ts:17-36` → `deliveries.service.ts:117-132` (no tenant argument) → `apps/api/src/infra/redis.module.ts:215-256` (`data: job.data`).
- Job payloads contain `tenantId`, `leadId`, `destinationId`, canonical lead data (`packages/types/src/jobs.ts` payloads).
- Blast radius: a `viewer` in tenant A can read tenant B's lead identifiers and payloads by paging `GET /deliveries/dead-letter?queue=delivery`.
- Related: `GET /deliveries/queue-stats` exposes **global** (all-tenant) queue counters — aggregate-only, lower severity, but still tenant-unscoped visibility.
- **Fix direction (not implemented):** filter `job.data.tenantId === actor.tenantId` (drop non-matching jobs server-side) or exclude `data` from responses entirely; tag queue jobs with tenant and make stats tenant-aware, or move queue stats behind an operator-only concept.

H-2 · HIGH — API key with default empty scopes becomes wildcard super-admin

- Evidence: `common/guards/permissions.guard.ts:31-32` honors `permission === "*"` as match-all; `common/guards/jwt-auth.guard.ts:140` maps **empty** `apiKey.scopes` → `permissions: ["*"]`; `packages/validation/src/index.ts:272-276` `apiKeyCreateSchema` defaults `scopes` to `[]`.
- Result: `POST /api-keys {"name":"x"}` (no scopes field) mints a key satisfying **all 73 permission-guarded routes**. The web UI's create dialog does not send scopes either.
- The wildcard is unreachable through roles (roles validate against the 34-code catalog: `roles.service.ts:212-217`) — this hole exists only on the API-key path.
- **Fix direction:** require non-empty scopes (explicit admin = `permissions` list or an explicit `"*"` opt-in), or default scopes to a minimal read set. Treat any existing empty-scope key as incident-worthy.

H-3 · MEDIUM-HIGH — Access tokens survive revocation

- Evidence: JWT path in `common/guards/jwt-auth.guard.ts:69-108` verifies signature and reloads the user, but **never consults** `UserSession`; logout/refresh-reuse revoke only sessions (`modules/auth/auth.service.ts:213-218, 244-247, 255-263`).
- Impact: after "logout everywhere" (or refresh-token-reuse mass revocation), a stolen access token keeps working until `JWT_ACCESS_TTL` (default **900 s / 15 min**).
- Disabled **user** and suspended **tenant** are checked per-request ✅ — the gap is session-level revocation only.
- **Fix direction:** check `UserSession` (jti/sid claim) on the JWT path, or accept the 15-minute window explicitly (document it).

H-4 · MEDIUM — Disabled user's API keys keep working

- Evidence: key path `jwt-auth.guard.ts:110-145` validates the key row (status/revoked/expires/tenant) but **never loads the owning user**; a DISABLED user's keys remain fully usable.
- **Fix direction:** join user status on key auth (or cascade `apiKey.status` when the user is disabled).

H-5 · MEDIUM — `POST /leads` idempotency depends entirely on an optional field

- Evidence: `Idempotency-Key` header is never read anywhere (grep = 0; only CORS/inbound allow-lists mention it). Dedup relies on `@@unique([tenantId, sourceAccountId, externalLeadId])` (`prisma/schema.prisma:450`); `externalLeadId` is optional (`leads.dtos.ts:11`). Postgres treats NULLs as distinct → retried POSTs create duplicate leads.
- Contrast: webhook ingress **is** properly idempotent (below), and outbound delivery uses real `Idempotency-Key` + DB unique (`worker/delivery/delivery.service.ts:130,180`).
- **Fix direction:** read `Idempotency-Key` on `POST /leads` (store on the lead or an idempotency table), or require `externalLeadId` for API-created leads.

H-6 · LOW-MEDIUM — Webhook contract deviations (status code + optional signature)

- **Status:** code comments document "respond 202" (`webhooks.service.ts:20-23`) but no `@HttpCode` exists anywhere; ingress returns **201** on first *and* duplicate deliveries.
- **Signature:** verification runs only **if the account has a secret** (`webhooks.service.ts:53-63`); secrets are auto-created only for `WEBHOOK|API|CUSTOM` source types (`sources.service.ts:170-181`). For other types without a secret, unsigned traffic is accepted unless `requireSignature: true` is set per-account (nothing sets it by default).
- What's good: HMAC-SHA256 over **raw body** (`main.ts:19`), timing-safe compare (`shared/crypto.ts:88-97`), header auto-detect (`x-signature` / `x-hub-signature-256` / `x-middleware-signature`), 600/min ingress throttle, DB idempotency unique.
- **Fix direction:** default `requireSignature=true` for all accounts with secrets, create secrets for every webhook-bearing source type, and either return 202 or correct the documented contract.

H-7 · LOW — Uneven audit coverage on mutations

Audited ✅ (~40 sites): leads create/replay/delete, sources CRUD+accounts+rotate, destinations CRUD+credential+test, mappings versions/activate, routing, users, roles, api-keys, imports create+download, tenant rename, delivery replay, scoring/dedup, auth register/login/failure/reuse.

Not audited ❌:

- `POST /auth/logout` (session revocation) — `auth.controller.ts:91-112`.
- `POST /auth/refresh` rotation — audited only on reuse detection (`auth.service.ts:219`).
- `GET /automation/dedup` **writes** the default config row on first read (`automation.service.ts:169-173`) — a mutation on a `dedup:read` path with no audit row.
- `POST /webhooks/inbound` creates WebhookEvent+Lead — acceptable (event table is the trail), but no AuditLog actor row.

H-8 · MEDIUM — Unbounded list responses

Cursor pagination (`take: limit+1`, default 25/max 100 via `shared/pagination.ts`) is correctly implemented on **9** lists: leads, deliveries, audit-logs, imports, users, api-keys, field-mappings, webhook-events (+ cursorQuerySchema consumers).

No `take` at all:

Endpoint	Evidence
GET /sources	`sources.service.ts:51-57`

Endpoint	Evidence
GET /sources/:id/accounts	`sources.service.ts:140-143`
GET /destinations	`destinations.service.ts:32-39`
GET /roles	`roles.service.ts:60-64`
GET /routing-rules	`routing-rules.service.ts:65-69`
GET /automation/scoring	`automation.service.ts:58-62`

Plus computational hazards: `GET /analytics/destinations` loads **every** tenant delivery row into JS (`analytics.service.ts:166-169`); `analytics/sources` lists all tenant sources (`:137-145`).

Risk: slow queries + memory growth as tenants grow; mitigating factor is per-tenant scoping (not cross-tenant) and that these are configuration-sized in practice — except scoring versions and analytics.

H-9 · LOW — Response data hygiene

Item	Where	Note
`storageKey` + `checksum` leaked	`GET /imports/:id` returns full row (`imports.service.ts:204-210`) while the list view deliberately excludes them (`:19-33`)	Storage key reveals internal bucket/dir layout
Refresh token in JSON body	register/login/refresh (`auth.controller.ts:16-21,153-160`) **in addition to** httpOnly cookie	Cookie alone is the safer pattern; body copy widens XSS exposure
Raw driver/Redis error strings	`GET /health` (`health.controller.ts:33,39`)	Version/connection details to unauthenticated callers
Unsanitized lead payloads	`GET /leads/:id/payloads` (`leads.service.ts:155-166`) — webhook *events* are sanitized, raw payloads are not	Inherits whatever the client sent; tenant-only access, so accepted risk — document
429 message passthrough	`all-exceptions.filter.ts:71-89` forwards `HttpException.message` verbatim	Controlled vocabulary today; keep messages free of internals

Verified safe ✅: password hashes excluded by select (`users.service.ts:9-17`), API key hash never returned (`api-keys.service.ts:8-19`), raw key **once** at create, destination credentials `stored:true` only, webhook secrets once/rotate-only, audit metadata secret-stripped (`shared/sanitize.ts`), no stack traces in any response (filter logs only, `all-exceptions.filter.ts:33-40`).

H-10 · LOW — Health endpoint semantics

`GET /api/v1/health` returns **HTTP 200** with `status:"degraded"` when PG/Redis fail (`health.controller.ts:41-47`) and includes raw error strings. It checks PG + Redis but **not the worker/BullMQ consumer** (no worker heartbeat anywhere; worker has no HTTP server). Orchestrators keyed on status codes would keep routing traffic to a broken instance.

Verification matrix (per §10 checklist)

Check	Result
Authentication	Dual: `Authorization: Bearer` (JWT, signature+issuer, DB-reloaded user, ACTIVE status, tenant match `:88-94`) or `x-api-key` (SHA-256 lookup, status/revoked/expiry/tenant `:112-123`). 6 `@Public` routes: register/login/refresh/logout, webhook ingress, health — all still throttled. Gap: H-3 session revocation, H-4 key-vs-user.
Authorization	73/78 non-public routes have `@RequirePermission`; the 2 permission-free authenticated routes (`GET /auth/me`, `GET /tenants/me`) are self-scoped by design; `POST /auth/logout` needs only possession of a refresh token (acceptable). Permissions resolved **per-request from DB role** (`jwt-auth.guard.ts:84-87,103`) — no stale JWT claims. Gap: H-2 wildcard keys.
Tenant isolation	`tenantId` is **never client-controlled** (grep: no body/query/header tenant anywhere; DTOs contain no tenant field; `CurrentUser` reads only `request.actor`). All list queries filter `tenantId`; all cross-tenant id lookups return **404** (anti-enumeration). Write paths use check-then-act (read scoped row → mutate by id) — **partial** TOCTOU pattern in 17 services (no `$transaction`; the provided `assertSameTenant` helper in `packages/database/src/tenant.ts:16` is unused, grep=0). No confirmed cross-tenant *write*; the confirmed cross-tenant *read* is H-1.
Input validation	Zod DTOs via `createZodDto` + `ZodValidationPipe` (`main.ts:59`) on all typed routes; body limit 1 MB; upload cap 50 MB. **Untyped:** `/webhooks/events` query (inline TS), `/deliveries/failed|dead-letter` query (raw strings, manual `Number()`), webhook ingress body (`unknown` — required for HMAC), inline `daysQuery` (validated in-handler).
Rate limiting	Global 300/min/IP; auth register/login/refresh 20/min; webhook ingress 600/min; no skips. 429 → `RATE_LIMITED`. Note: `/auth/logout` falls under global limit (fine). Health unauthenticated under 300/min (probe-friendly).
Sensitive data	See H-9. At rest: bcrypt cost 12, SHA-256 key hashes, AES-256-GCM credentials (`v1.<iv>.<tag>.<ct>`, `shared/crypto.ts:41-54`), refresh rotation with reuse detection.
Audit logging	See H-7. Fail-open by design (never breaks the request), append-only tables (no `updated_at` on AuditLog/LeadStageEvent/DeliveryAttempt — tested).
IDOR / cross-tenant	No bare-id IDOR found beyond H-1's non-id variant; every `:id` route is tenant-checked (evidence table per service in the working notes; spot-checked all 17 modules).
Privilege escalation	Roles can only grant catalog codes (34); system roles immutable; self-DISABLE refused; wildcard reachable only via H-2. No endpoint accepts `role` / `permissions` / `tenantId` for the actor themselves outside `users:write` (which is itself permissioned).
Credential exposure	None in responses except one-time reveals (API key, webhook secret) — by design.

Explicitly tested behaviors that back these claims (existing test suite)

- Tenant isolation semantics: `tests/integration/tenant-isolation.test.ts` (service-level, 404 semantics, raw-query scoping).
- DB idempotency uniques (webhook events, leads, deliveries, attempts): `tests/integration/idempotency.test.ts`.
- Replay immutability/attempt numbering: `tests/integration/replay.test.ts`.
- Signed-webhook e2e + duplicate collapse + bad-signature rejection: `tests/queue/pipeline-e2e.test.ts`.
- **Not tested at HTTP level:** guards, permission denials, rate limits, error envelopes — no supertest/controller tests exist (inventory Appendix 2).

Recommended remediation order (approval required before any change)

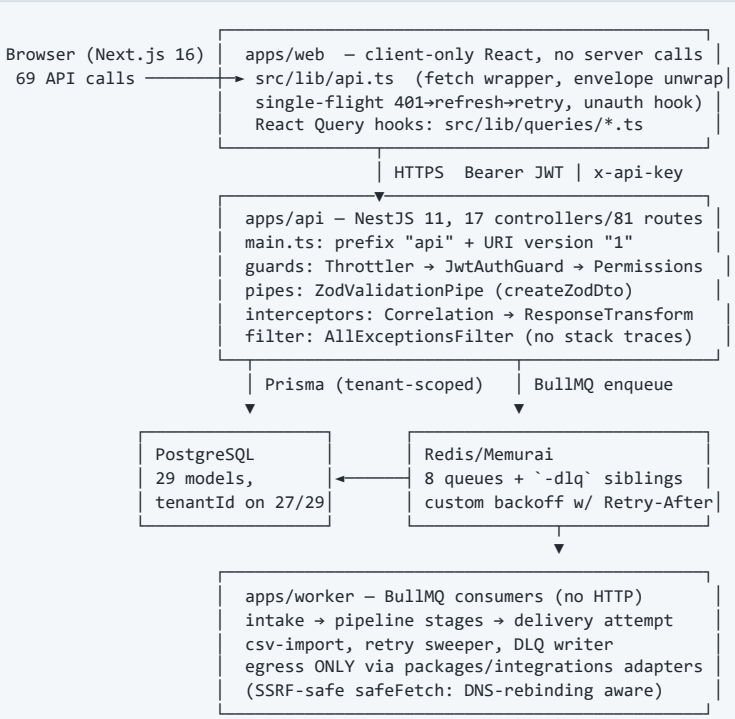
1. **H-1** — scope queue endpoints (blocking for any multi-tenant exposure).
2. **H-2** — forbid implicit wildcard API keys; audit existing keys in DB.
3. **H-4** — tie key auth to user status (one-line join).
4. **H-6** — `requireSignature` default + webhook status-code decision.
5. **H-3** — session check on JWT path (or documented 15-min revocation window).
6. **H-8/H-9/H-10** — bounds, storageKey/refresh-token hygiene, health 503 semantics.
7. **H-5/H-7** — POST /leads idempotency; audit gaps.

No findings required schema changes except optionally an idempotency table for H-5 and (already-planned) password-reset tokens.

API Architecture — Lead Integration & Distribution Middleware

Audit date: 2026-10-01 · Describes the system **as built** (verified against code) and the recommended target state. No changes were made.

1. System shape



Key facts:

- **Single HTTP surface.** One Nest app; no microservices, gateways, SSE, or worker HTTP (`apps/worker` makes zero HTTP calls of its own — all egress through adapters).
- **No Next.js server-side proxy.** No `route.ts` handlers, no server actions, no rewrites (`next.config.ts` is 7 lines). The browser talks to the API directly with `NEXT_PUBLIC_API_URL`.
- **Zod all the way down.** `packages/validation` (34 schemas) is the single source of request truth; controllers wrap schemas with `nestjs-zod`'s `createZodDto`. Shared enums/contracts in `packages/types` mirror Prisma enums 1:1.

2. Request lifecycle (contract)

1. **Correlation** — `x-correlation-id` accepted (validated `^[A-Za-z0-9._:-]{8,128}$`) or generated; echoed on the response (`request-context.middleware.ts`).
2. **Rate limit** — ThrottlerGuard first (applies even to `@Public` routes).
3. **Authenticate** — `JwtAuthGuard`: `x-api-key` wins if present (SHA-256 lookup) else Bearer JWT; every request reloads the principal from DB (user status, tenant status, role permissions fresh — no stale claims). Sets ambient ALS actor `{userId, tenantId, permissions…}`.
4. **Authorize** — `PermissionsGuard` reads `@RequirePermission(...)` metadata; `"*"` matches all (currently reachable only via empty-scope API keys — security H-2).
5. **Validate** — `ZodValidationPipe` on body/query/params DTOs → 400 `BAD_REQUEST` + `details[]` on failure (service-thrown `ZodError` → 422 `VALIDATION_FAILED`).
6. **Handle** — controller → service. Services take `actor` first; every read filters `tenantId`; writes follow check-then-act on a tenant-scoped row.
7. **Audit** — mutations call `audit.record` (~40 sites; secret-stripped; fail-open).
8. **Respond** — `ResponseTransformInterceptor` wraps `{success:true,data}` (`@SkipTransform` for CSV download); errors → `{success:false,error:{code,message,requestId,details?}}`.

Status codes in use: POST→201 (no `@HttpCode` anywhere), everything else→200; 400/401/403/404/409/422/429/500/503 from the shared `ERROR_CODES` map (`packages/types/src/errors.ts`). Health returns 200-even-when-degraded (finding H-10).

---

3. Authentication model

Aspect	Implementation
Access token	JWT HS256, `JWT_ACCESS_TTL` 900 s, issuer `lead-middleware`; **not** stateful — validated by signature + DB principal reload
Refresh token	SHA-256-hashed in `UserSession`, httpOnly cookie scoped `/api/v1/auth` (+ returned in body — H-9), **rotating**; reuse detection revokes all sessions
API keys	`x-api-key`, SHA-256 `keyHash` unique; status/revoked/expiry checked; `lastUsedAt` fire-and-forget
Permissions	34-code catalog (`packages/types/src/permissions.ts`); role-permissions re-read per request; 4 seeded system roles
Webhooks	Unauthenticated by token: identified by unguessable `publicId` + HMAC over raw body
Known gaps	H-3 revocation window, H-4 disabled-user keys, H-2 wildcard scopes (see security doc)

Tenant derivation: credential → `actor.tenantId` exclusively. Webhook: `publicId → LeadSourceAccount.tenantId`. Nothing accepts tenantId from request input.

---

4. Data & idempotency model

Concern	Mechanism
Tenant scoping	`tenantId` on 27/29 models (exceptions: `Tenant`, `UserSession` global; `AuditLog.tenantId` nullable-by-design for SYSTEM/WORKER actors)
Inbound webhook idempotency	`@@unique([tenantId, sourceAccountId, idempotencyKey])`; key = external event id or body hash; duplicate → resume/ack
Manual lead idempotency	`@@unique([tenantId, sourceAccountId, externalLeadId])` — **optional field ⇒ gaps when omitted (H-5)**
Delivery identity	`@@unique([tenantId, leadId, destinationId])`; attempts `@@unique([tenantId, deliveryId, attemptNumber])` — append-only, never rewritten
Replay semantics	Global attempt counter; each replay opens a cycle (`replayCycle`, `cycleStartAttempt`, `maxAttempts += 5`); cycle-local backoff; P2002 treated as invariant guard
Outbound idempotency	Worker sends `Idempotency-Key: tenantId|leadId|destinationId` to destinations
Immutable history	`DeliveryAttempt`, `LeadStageEvent`, `LeadScore`, `LeadRawPayload`, `AuditLog` — no `updated_at` columns (tested)
Secrets at rest	bcrypt(12), SHA-256 hashes, AES-256-GCM `v1.<iv>.<tag>.<ct>`

5. Queue topology (BullMQ)

Queue	Producer(s)	Consumer	DLQ
`lead-intake`	API (webhook fast-ack, manual POST /leads), worker (CSV)	`intake.processor` → chains lead-processing	`lead-intake-dlq`
`lead-processing`	intake processor, API lead replay	`pipeline.processor` (stage chain)	`lead-processing-dlq`
`enrichment` / `scoring` / `routing`	**none (stage queues currently dead — processed inline by pipeline)**	registered processors	`*-dlq`
`delivery`	pipeline `enqueueDelivery`, replay sweeper	`delivery.service.attempt()` (concurrency 10)	`delivery-dlq` (terminal `UnrecoverableError`)
`retry`	API `POST /deliveries/:id/replay` + repeatable `sweep-deliveries`	`replay.processor` / sweeper	`retry-dlq`
`csv-import`	API `POST /imports`	`csv-import.processor` (100k rows, deterministic externalLeadId)	`csv-import-dlq`

- Backoff: custom strategy, base 2 s, max 300 s, ±10 % jitter, honors destination `Retry-After` (`queues.ts:60-81`, `workers.service.ts:130-136`); DB-level caps enforce attempt ladders (2/4/8/16 s tested).
- DLQ: worker `failed` handler → `<queue>-dlq` with `originalJobId/failedReason/attempts`, retained forever (`removeOnComplete/Fail:false`).
- **Note:** dead `enrichment` / `scoring` / `routing` queues exist but nothing enqueues them (pipeline executes stages in-process). Either wire producers or retire them — flagged for architecture review, not an API issue.
- Queue admin APIs are intentionally **absent** (safety requirement).

---

6. Egress & adapters

- All outbound traffic (delivery + connection tests) goes through `packages/integrations` registry: `salesforce, zoho, hubspot, pipedrive, dct-crm, custom-api, webhook` — covers all 7 `DestinationType`s.
- `safeFetch` (`http.ts:107-143`): http/https only, no embedded credentials, literal private-IP block, **DNS `lookup(host,{all:true})` rejects if any resolved address is private** (rebinding-safe), hard timeout, sanitized ≤2 KB error bodies, `DESTINATION_SSRF_BLOCKED` code.
- Outbound webhook adapter signs with its own HMAC (`x-middleware-signature` + timestamp).
- Test hook: `POST /destinations/:id/test` exercises the same guarded path.

---

7. Frontend ↔ API mapping (coverage health)

- **69 distinct calls, 73 call sites — 100 % matched to real routes** (0 calls to missing endpoints, 0 wrong HTTP methods). All calls originate in `lib/api.ts` + `lib/queries/*`.
- List envelopes `{items,nextCursor,total}` are supersets of what the web reads; web uses `useInfiniteQuery` cursors consistently.
- Unconsumed-by-web (11): all 4 `/imports`, 2 `/webhooks/events`, `DELETE /leads/:id`, `GET /sources/:id`, `PATCH /sources/accounts/:accountId`, `GET /destinations/:id/health`, `POST /auth/register` (see inventory §E — all intentionally kept).
- Mismatches to fix (inventory §I): dead `useCreateLead` wrong body; `{queued}` vs `{replayed}` type; `retryAt` shows job creation time.

---

8. Testing posture

Layer	Status
Unit (vitest)	4 files — worker pipeline, conditions, CSV, queue jobId
Integration (real PG)	6 files — tenant isolation, DB idempotency, pipeline, replay, audit, db-safety
Queue/e2e (real Redis)	queue mechanics + signed-webhook→delivery e2e
HTTP surface (supertest)	**none — zero controller/guard/DTO tests**

Layer	Status
Playwright	script exists, no config/tests

Highest-leverage additions: auth flows (401/403 matrix), RBAC denial per permission family, cross-tenant 404 matrix over HTTP, rate-limit 429, webhook signature + 201/202 contract, `POST /leads` idempotency, upload validation.

---

9. Documentation & OpenAPI posture

- `@nestjs/swagger` v11 installed and `@ApiTags/@ApiOperation` present on **all 17 controllers** — but **no `SwaggerModule.createDocument` call exists**, so no UI/document is ever served; `@ApiProperty` unused (zod DTOs instead).
- Repo previously contained **no docs at all** (no README, no docs/, no openapi/postman artifacts).
- This audit created `docs/api-inventory.md`, `docs/api-gaps.md`, `docs/api-security-audit.md`, this file, and `docs/openapi.yaml` (hand-derived from code, marked `x-audit-status`).

---

10. Recommended architecture (J)

Direction: fix and harden the current design — it is coherent, typed, tested at the data layer, and nearly complete. Do not re-platform.

1. **Security first (blocking for multi-tenant exposure):** implement H-1 queue scoping, H-2 wildcard-key removal, H-4 key/user status coupling — all guard/service-level, zero schema changes.
2. **Finish the two half-built features:** `POST /auth/change-password` (schema exists), `POST /imports/:id/cancel` (enum+worker exist).
3. **Make OpenAPI a build artifact:** register `SwaggerModule` with `DocumentBuilder` in `main.ts` (dependency already present; decorators already present) so `docs/openapi.yaml` stops being hand-maintained. Keep `x-audit-status` separation (implemented/planned/missing) until then.
4. **Contract coherence pass:** webhook 202 vs 201 (pick one and align code+docs), health 503 on critical failure + worker heartbeat (smallest viable: worker writes a heartbeat row/Redis key health can read), `retryAt` real semantics (BullMQ `nextProcessedAt`), zod DTOs for the three untyped query surfaces.
5. **Bound every list:** add `take` to the 6 unbounded reads (or explicitly document as bounded reference data with hard caps); convert `analytics/destinations` to SQL `groupBy`.
6. **Idempotency story:** honor `Idempotency-Key` on `POST /leads` (or require `externalLeadId`); keep webhook/delivery model as-is.
7. **Sequenced feature gaps (from api-gaps.md):** REST completeness (users detail/delete, audit detail) → password reset (after mailer) → import preview / pipeline simulate (shared read-only evaluator for routing+scoring+dedup) → enrichment provider domain → OAuth credential vault → tenants-CRUD admin console. Each gated on an approved consumer.
8. **Test the HTTP surface** (supertest matrix above) before any external exposure; keep queue/admin routes unexposed.
9. **Naming cleanups only when touching those routes anyway:** nest `/sources/accounts/:accountId` under `/sources/:id/accounts/:accountId`, keep `/automation/*` grouping as the precedent for future feature groups.
10. **Retire or wire the three dead stage queues** (`enrichment`, `scoring`, `routing`) to match reality.

Non-goals (confirmed absent on purpose): queue administration APIs, duplicate-merge semantics, bulk lead mutation, unversioned routes, per-provider webhook handlers, tenantId in requests.